STREAMIFY

Video Streaming & Live Broadcasting Platform

Product Requirements Document

Version 1.0 · May 2026
Status: Draft

Document Information

Field	Details
Document Title	Streamify — Product Requirements Document
Version	1.0
Status	Draft — Under Review
Author(s)	Engineering Team
Date	May 2026
Reviewers	Product, Engineering, DevOps

1. Product Overview

1.1 Vision

Streamify is a full-stack video streaming and live broadcasting platform designed for creators and learners. It provides YouTube-like video-on-demand (VOD) capabilities combined with Twitch-like live streaming — built on a production-grade microservices architecture.

The platform is engineered to scale horizontally, handle concurrent live streams, deliver adaptive-bitrate video globally, and serve as the primary engineering learning project demonstrating distributed systems, DevOps, and cloud infrastructure best practices.

1.2 Problem Statement

Engineers learning distributed systems lack a realistic, production-grade project that combines:

- Event-driven microservices with Kafka
- Real-time media processing (RTMP ingest, HLS delivery)
- Polyglot architecture (TypeScript + C#)
- Cloud-native deployment on Kubernetes
- Observable, production-ready infrastructure

1.3 Goals

- Build a working YouTube/Twitch-like product with real engineering depth
- Demonstrate each microservice boundary clearly and justify its existence
- Produce a deployable system on AWS with proper CI/CD, monitoring, and security
- Serve as a reference architecture for senior backend engineering skills

1.4 Non-Goals (v1.0)

- Monetization, ads, or payment processing
- Mobile native apps (web-only for v1.0)
- DRM / content protection beyond signed URLs
- Multi-region active-active deployment

- Recommendation algorithm / ML features

2. System Architecture

2.1 Architecture Style

Streamify uses a microservices architecture with event-driven communication via Kafka. Services communicate synchronously via REST/HTTP for request-response flows and asynchronously via Kafka for events that cross service boundaries. All services live in a single Nx monorepo.

2.2 Services

Service	Runtime	Storage	Responsibility
api-gateway	NestJS (TS)	Redis	JWT validation, rate limiting, request routing
user-service	NestJS (TS)	PostgreSQL	Registration, auth, profiles, stream keys
video-service	NestJS (TS)	PostgreSQL + S3	Upload, metadata, rendition management
stream-service	NestJS (TS)	Redis	RTMP key validation, viewer counts, chat pub/sub
transcode-worker	C# .NET 8	S3	FFmpeg worker, HLS segment generation
comment-service	NestJS (TS)	PostgreSQL	Video comments, threaded replies
notification-svc	NestJS (TS)	Redis	Push / email notifications via Kafka consumer
analytics-service	NestJS (TS)	PostgreSQL	View counts, watch-time, live metrics
search-service	NestJS (TS)	Elasticsearch	Full-text video/channel search

2.3 Infrastructure Components

Component	Purpose
Apache Kafka	Async event bus. Topics: video.uploaded, transcode.done, stream.started, stream.ended, user.events
Redis 7	JWT refresh token store, rate limit counters, stream viewer counts, pub/sub for live chat
PostgreSQL 16	Per-service databases (never shared). Managed via Prisma migrations per service
MinIO / S3	Object storage for raw video uploads, HLS segments (.ts files), manifests (.m3u8)
Elasticsearch 8	Full-text search index for videos and channels
Node-Media-Server	RTMP ingest server. Validates stream keys via webhook to stream-service
CloudFront CDN	HLS segment delivery, origin = S3. Signed URLs for private content

3. Kafka Event Contracts

All cross-service communication happens via strongly-typed Kafka events defined in the shared library. Events are the public API of each service — treat them as immutable contracts once deployed.

3.1 Event Topics

Topic	Producer	Consumers
video.uploaded	video-service	transcode-worker, analytics-service
transcode.done	transcode-worker	video-service (status update), notification-svc, search-service
transcode.failed	transcode-worker	video-service, notification-svc
stream.started	stream-service	notification-svc, analytics-service
stream.ended	stream-service	analytics-service, video-service (VOD save)
user.events	user-service	notification-svc, all services caching user info

3.2 Event Schema Example — video.uploaded

All events include a base envelope. Services must handle unknown fields gracefully (forward compatibility).

```
{ "videoId": "uuid", "userId": "uuid", "s3Key": "raw/uuid.mp4", "originalFilename": "talk.mp4",
"sizeBytes": 104857600, "timestamp": "2026-05-12T10:00:00Z" }
```

4. Features & Requirements

4.1 Authentication & Identity

Functional Requirements

- Register with email + password. Email verification required before publishing
- Login returns a short-lived JWT access token (15 min) and a long-lived refresh token (30 days)
- Refresh tokens stored in Redis with rotation on use — old token invalidated immediately
- Users can manage up to 3 active sessions; device list visible in settings
- Stream key generation: cryptographically random 32-byte hex key, regeneratable at any time
- OAuth2 social login (Google) — v1.1 scope

Non-Functional Requirements

- Auth endpoints rate-limited: 5 failed attempts → 15-minute lockout per IP
- Passwords hashed with bcrypt, cost factor 12
- All JWT validation at API Gateway — downstream services trust x-user-id header

4.2 Video Upload & VOD

Functional Requirements

- Upload via multipart form up to 10 GB per file
- Supported input formats: MP4, MOV, AVI, MKV, WebM
- S3 presigned URL used for direct upload (no proxying bytes through Node.js)
- Video status lifecycle: UPLOADING → PROCESSING → READY | FAILED
- Transcode to three renditions: 360p, 720p, 1080p as HLS (.m3u8 + .ts segments)
- Segment length: 6 seconds. Playlist type: VOD
- Thumbnail auto-generated from frame at 10% of duration; manual override allowed
- Video metadata: title (required), description, tags, category, visibility (public/unlisted/private)
- Creators can delete their videos; raw file + all renditions removed from S3

Transcoding (C# Worker)

- Worker consumes video.uploaded from Kafka consumer group "transcode-workers"
- Max 2 concurrent transcodes per worker instance (configurable via env)
- Reports progress events every 10% via Kafka topic transcode.progress
- Idempotency: checks video status before starting — skips if already PROCESSING
- On failure: retries up to 3 times with exponential backoff, then emits transcode.failed

4.3 Live Streaming

Functional Requirements

- RTMP ingest via Node-Media-Server at rtmp://ingest.streamify.io/live/{stream_key}
- Stream key validated via webhook: stream-service verifies key and returns user context
- FFmpeg spawned per stream: RTMP → HLS segments → S3 (low-latency LL-HLS target 3s)
- Playback URL served via CDN: https://cdn.streamify.io/live/{channelId}/index.m3u8
- Viewer count tracked in Redis INCR/DECR, published to client via SSE
- Live chat: Redis pub/sub per stream, SSE delivery to viewers
- Stream title and category settable via API before / during stream
- Automatic stream.ended event if no RTMP data received for 30 seconds
- VOD archive: after stream ends, assembled into full video via video-service

Non-Functional Requirements

- Max end-to-end latency (LL-HLS): < 5 seconds
- Node-Media-Server scales horizontally; stream routing via Redis stream registry

4.4 Playback

- Video.js player with HLS.js for adaptive bitrate streaming
- Quality selector: auto (ABR) + manual 360p / 720p / 1080p
- Resume playback: position saved per user per video in localStorage + server-side for logged-in users
- Keyboard shortcuts: space (play/pause), f (fullscreen), left/right arrows (±10s), m (mute)
- Picture-in-picture supported

4.5 Discovery & Search

- Homepage: recommended videos (recently uploaded, trending by view count)
- Search: full-text over title, description, tags via Elasticsearch
- Search indexed on transcode.done event — only READY videos are indexed
- Filters: date, duration, category
- Autocomplete suggestions (Elasticsearch completion suggester)

4.6 Creator Studio

- Video manager: list, edit metadata, delete, view analytics per video
- Analytics: view count over time, average watch duration, traffic sources
- Live dashboard: current viewer count, stream health (bitrate, dropped frames)
- Comment moderation: hold for review, delete, block users

5. Core Data Models

Each service owns its database. Cross-service references use UUIDs without foreign key constraints. Denormalized data (e.g., channel name on video) is synchronized via Kafka events.

5.1 User Service Schema (PostgreSQL)

Column	Type	Notes
id	UUID PK	gen_random_uuid()
email	TEXT UNIQUE	Lowercase, indexed
password_hash	TEXT	bcrypt cost 12
username	TEXT UNIQUE	URL-safe, 3–30 chars
stream_key	TEXT UNIQUE	32-byte random hex, nullable if not set
verified_at	TIMESTAMPTZ	NULL until email verified
created_at	TIMESTAMPTZ	DEFAULT NOW()

5.2 Video Service Schema (PostgreSQL)

Column	Type	Notes
id	UUID PK	Primary key
user_id	UUID NOT NULL	No FK — references user service
title	TEXT NOT NULL	Max 150 chars
description	TEXT	Max 5000 chars
status	TEXT NOT NULL	UPLOADING|PROCESSING|READY|FAILED

Column	Type	Notes
visibility	TEXT NOT NULL	public|unlisted|private
raw_s3_key	TEXT	Set on upload, cleared after READY
duration_seconds	INT	Populated by transcode worker
view_count	BIGINT DEFAULT 0	Incremented by analytics service
created_at	TIMESTAMPTZ	DEFAULT NOW()

5.3 Video Renditions

Column	Type	Notes
id	UUID PK	
video_id	UUID	FK → videos.id
quality	TEXT	360p | 720p | 1080p
playlist_key	TEXT	S3 key for .m3u8 manifest
segment_prefix	TEXT	S3 prefix for .ts chunks

6. API Reference (v1)

Base URL: https://api.streamify.io/v1 All authenticated routes require Authorization: Bearer <access_token>

6.1 Authentication

Method	Path	Description
POST	/auth/register	Register user. Returns 201 with userId
POST	/auth/login	Returns { accessToken, refreshToken }
POST	/auth/refresh	Rotate refresh token. Invalidates old one
POST	/auth/logout	Revoke refresh token from Redis

6.2 Videos

Method	Path	Description
POST	/videos/upload-url	Returns S3 presigned PUT URL and videoId
PATCH	/videos/:id/metadata	Update title, description, tags, visibility

Method	Path	Description
GET	/videos/:id	Video detail + rendition URLs (signed)
GET	/videos	List public videos. Supports ?page, ?limit, ?category
DELETE	/videos/:id	Delete video + all S3 assets (creator only)
GET	/videos/:id/status	Polling endpoint for transcode status

6.3 Live Streams

Method	Path	Description
GET	/streams	List active live streams
GET	/streams/:channelId	Stream detail + HLS playback URL
GET	/streams/:channelId/viewers	SSE endpoint: viewer count updates
GET	/streams/:channelId/chat	SSE endpoint: live chat messages
POST	/streams/:channelId/chat	Send chat message (authenticated)
POST	/users/me/stream-key/regenerate	Generate new stream key, invalidates old

7. Non-Functional Requirements

7.1 Performance

Metric	Target
API Gateway p99 latency	< 100ms (excluding transcode wait)
Video playback start time (CDN hit)	< 2 seconds
Live stream end-to-end latency (LL-HLS)	< 5 seconds
Search response time p95	< 300ms
Concurrent live streams supported	50 per Node-Media-Server instance
Concurrent API requests (v1.0)	1,000 rps per service replica

7.2 Reliability

- Kafka producers use acks=all for critical events (video.uploaded, transcode.done)
- Transcode worker is idempotent — safe to retry on duplicate Kafka delivery
- S3 multipart uploads with client-side retry for network failures

- Database: connection pooling via PgBouncer. Max pool size configurable per service
- Health check endpoints at /health (liveness) and /ready (readiness) on all services
- Circuit breaker pattern for inter-service HTTP calls (opossum library in NestJS)

7.3 Security

- JWT access tokens: HS256, 15-minute TTL. Refresh tokens: 30-day TTL, Redis-backed
- All inter-service traffic on internal Kubernetes network only — not exposed externally
- RTMP stream key treated as secret: never logged, stored hashed in Redis for lookup
- S3 objects not publicly readable — all access via presigned URLs or CloudFront signed URLs
- Rate limiting: 100 req/min per IP on public routes, 1000 req/min per authenticated user
- Input validation with class-validator on all NestJS endpoints
- SQL injection prevention: Prisma parameterized queries only — raw SQL prohibited
- CORS: whitelist of allowed origins, no wildcard in production
- Secrets managed via Kubernetes Secrets + AWS Secrets Manager (never in env files)

7.4 Scalability

- Stateless services: all state in Redis/PostgreSQL — horizontal scaling with zero config changes
- Kafka consumer groups: transcode-workers group allows adding workers without reconfiguring producers
- HPA (Horizontal Pod Autoscaler): CPU > 70% triggers scale-out, minimum 2 replicas per service
- CDN absorbs video playback load — origin (S3) is not hit on cache hits

7.5 Observability

- Structured JSON logs on all services (pino in NestJS, Serilog in C#)
- Distributed tracing: OpenTelemetry SDK on all services → Jaeger
- Metrics: Prometheus scraping all services. Grafana dashboards per service + cross-service
- Error tracking: Sentry in NestJS services and C# worker
- Kafka consumer lag monitored via Prometheus JMX exporter
- Alerting: PagerDuty integration for p99 latency breaches and error rate > 1%

8. DevOps & Deployment

8.1 Infrastructure

- Cloud: AWS (EKS for Kubernetes, RDS PostgreSQL, ElastiCache Redis, S3, CloudFront, MSK Kafka)
- IaC: Terraform for all AWS resources. State in S3 + DynamoDB lock
- Local development: docker-compose with all dependencies (Kafka, Redis, PostgreSQL, MinIO, Elasticsearch)

8.2 CI/CD Pipeline (GitHub Actions)

1. PR opened: lint, typecheck, unit tests, build Docker images
2. PR merged to main: integration tests, publish images to ECR with SHA tag
3. Tag pushed (v*.*.*): deploy to staging environment, run smoke tests
4. Manual approval gate: deploy to production with blue/green strategy

5. Post-deploy: synthetic monitor runs, alert if failure

8.3 Kubernetes

- Namespace per environment: streamify-dev, streamify-staging, streamify-prod
- Each service: Deployment + Service + HPA + PodDisruptionBudget
- Ingress: NGINX Ingress Controller with cert-manager (Let's Encrypt)
- Secrets: External Secrets Operator syncing from AWS Secrets Manager
- Resource requests/limits defined on all containers — no unbounded pods

8.4 Docker

- Multi-stage Dockerfiles: build stage + minimal runtime stage
- NestJS images: node:20-alpine base, non-root user, no dev dependencies in final image
- C# image: mcr.microsoft.com/dotnet/runtime:8.0-alpine, self-contained publish
- .dockerignore excludes: node_modules, .git, dist, coverage

9. Build Roadmap

Phase	Timeline	Deliverables
Phase 1	Weeks 1–3	Monorepo setup (Nx), User service (auth + JWT), API Gateway, local docker-compose
Phase 2	Weeks 4–6	Video service, S3 upload, Kafka wiring, C# transcode worker, basic playback
Phase 3	Weeks 7–9	Live streaming (RTMP ingest, LL-HLS, viewer count, chat via SSE)
Phase 4	Weeks 10–12	Comment service, search (Elasticsearch), analytics service, notification service
Phase 5	Weeks 13–16	Kubernetes deployment, CI/CD pipeline, Prometheus + Grafana, Jaeger tracing
Phase 6	Weeks 17–20	Terraform IaC for AWS, CloudFront CDN, production hardening, security audit
v1.1	Post v1.0	Google OAuth, DRM (signed URLs), recommendation engine, mobile web PWA

10. Risks & Mitigations

Risk	Likelihood	Mitigation
Kafka consumer lag under burst uploads	Medium	HPA on transcode-worker, consumer group lag alerting

Risk	Likelihood	Mitigation
FFmpeg OOM on large 4K input files	Medium	Container memory limits, stream-based processing, reject files > 10GB
RTMP server becomes single point of failure	High	Multiple NMS replicas, stream routing registry in Redis
S3 egress costs at scale	High	CloudFront in front of S3 reduces direct S3 requests by >95%
Inter-service circular dependency via Kafka	Low	Event ownership matrix enforced in code review
Refresh token theft / session hijack	Medium	Refresh token rotation, Redis TTL, anomaly detection on concurrent use

11. Glossary

Term	Definition
HLS	HTTP Live Streaming. Apple's adaptive bitrate protocol. Segments video into .ts chunks with .m3u8 playlist
LL-HLS	Low-Latency HLS. Extension of HLS targeting < 5s end-to-end latency using partial segments
RTMP	Real-Time Messaging Protocol. Used by OBS and encoders to push video to ingest servers
Presigned URL	Time-limited S3 URL granting access to a specific object without requiring AWS credentials
Consumer Group	Kafka mechanism allowing multiple workers to share processing of a topic's partitions
ABR	Adaptive Bitrate. Player automatically selects quality based on available bandwidth
HPA	Kubernetes Horizontal Pod Autoscaler. Scales replicas based on CPU/memory or custom metrics
IaC	Infrastructure as Code. Defining cloud infrastructure in version-controlled configuration files (Terraform)